Sorbonne Université

Master 1 AI2D

Fondements des Systèmes Multi-agents - FoSyMa

Rapport de projet DEDALE

Système multi-agents pour la chasse au golem

Groupe : 1

Yuxiang Zhang
Kenan Alsafadi

2025/2026

Dépôt Git : https://github.com/jamespaulzhang/FOSYMA-Groupe1-Zhang-Alsafadi

Table des matières

1 Introduction 2

2 Architecture générale du code 3

3 Agents et comportements 4

4 Choix d'exploration 5

5 Carte, chemins et partage incrémental 6

6 Communication 7

7 Coordination et blocage 8

8 Chasse au golem 9

9 Optimalité, coûts et critères d'arrêt 10

10 Conclusion 11

1 Introduction

Ce rapport présente le projet Dedale/JADE développé pour contrôler une équipe d'agents explorateurs dans un environnement graphé, inconnu au départ, où un(ou plusieurs) golem mobile doit être localisé puis bloqué collectivement. Le problème combine exploration distribuée, fusion de connaissances, communication partielle, décision locale et coordination tactique.

La solution repose principalement sur `FSMExploAgent`, un agent à états finis capable de passer de l'exploration à la chasse, puis au blocage. Chaque agent maintient une carte locale `MapRepresentation`, enrichie par des traces d'odeur et synchronisée avec les autres agents par des messages ACL. Le golem est représenté comme une entité mobile de type Wumpus.

L'objectif technique n'est pas seulement de trouver un chemin court : les agents doivent construire une vision commune assez fiable pour choisir des cibles, éviter une redondance excessive et fermer les issues autour du golem. Les choix faits dans le code privilégient donc des heuristiques réactives, compréhensibles et peu coûteuses, plutôt qu'une optimisation globale centralisée.

Table 1 – Axes traités dans le projet

Axe	Objectif traité dans le projet
Exploration	Découvrir les sommets et arêtes, fermer les sommets visités, rejoindre le plus proche sommet ouvert.
Communication	Partager positions, odeurs, golems connus, captures et deltas de carte avec messages ACL typés.
Coordination	Utiliser les positions alliées, les points d'articulation et un protocole CFP/AWARD/ACK pour affecter les blocages.
Chasse	Exploiter les odeurs, les détections directes et la structure de graphe pour poursuivre puis encercler le.s golem.s.

2 Architecture générale du code

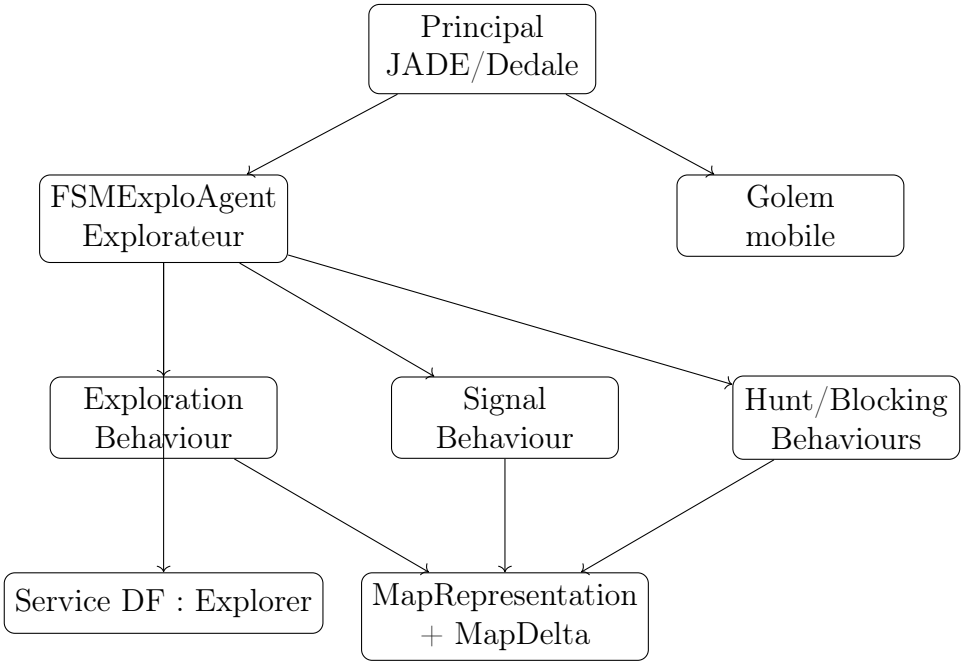

FIGURE 1 – Structure logique et principaux flux d'information.

Le point d'entrée `Principal` crée la plateforme JADE, le GateKeeper Dedale, les conteneurs et les agents décrits dans les fichiers JSON de `resources`. La configuration active utilise une topologie GraphStream et plusieurs agents explorateurs `FSMExploAgent`, plus un ou plusieurs golems.

Le code est organisé autour de quatre responsabilités : lancement de la plateforme, classes d'agents, comportements JADE, et connaissances de graphe. Cette séparation rend la logique lisible : les comportements décident, tandis que `MapRepresentation` concentre les opérations de graphe, de sérialisation, de calcul de chemin et de gestion des odeurs.

TABLE 2 – Responsabilités principales des packages et classes

Package ou classe	Responsabilité principale
`princ.Principal/ ConfigurationFile`	Initialisation JADE/Dedale, choix de la topologie, chargement des entités JSON.
`agents.dummies.explo. FSMExploAgent`	État global de l'agent : mode, destination, golems connus, captures, positions alliées, tâches de blocage.
`behaviours.*`	Unités d'action JADE : setup, exploration, signalement, chasse, négociation de blocage, maintien d'une position.
`knowledge. MapRepresentation`	Carte GraphStream locale, sommets ouverts/fermés, Dijkstra, odeurs, deltas, points d'articulation.
`knowledge.GolemInfo/ MapDelta`	Objets sérialisables échangés entre agents : état d'un golem et changements incrémentaux de carte.

3 Agents et comportements

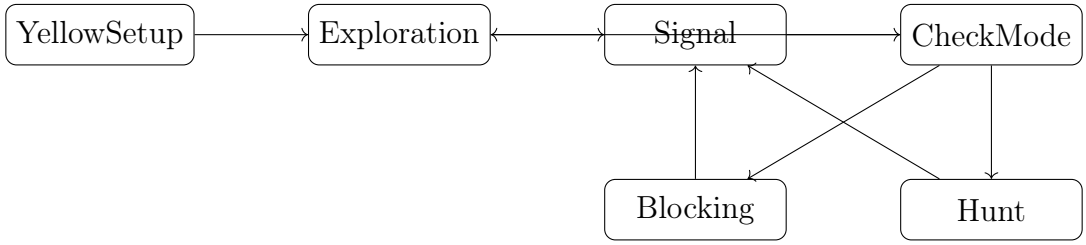

FIGURE 2 – Cycle comportemental de l'agent explorateur. Transitions : exploration terminée vers chasse ; affectation ; faire un blocage.

Le comportement global est un FSM. L'état `YellowSetup` enregistre l'agent dans le Directory Facilitator avec le service `Explorer`. Ensuite, chaque cycle alterne une décision métier, exploration, chasse ou blocage, et un passage par `SignalBehaviour`, qui intègre et diffuse les informations utiles.

Cette alternance est importante parce qu'elle évite qu'un agent accumule trop longtemps une décision locale sans écouter les autres. Après chaque mouvement ou observation, il partage sa position, ses odeurs, ses golems connus et les deltas de carte. Le comportement `CheckMode` relance ensuite l'état adapté selon le mode courant.

TABLE 3 – Rôle des comportements

Comportement	Rôle dans le système
`ExplorationBehaviour`	Observe, ferme le sommet courant, ajoute voisins et arêtes, choisit un sommet non visité ou le plus proche ouvert.
`SignalBehaviour`	Traite les protocoles ACL : positions, odeurs, golems, captures, deltas de carte, CFP et awards.
`HuntBehaviour`	Sélectionne une cible, suit l'odeur, détecte les golems visibles, lance la coordination de blocage.
`BlockerBehaviour`	Rejoint ou maintient le sommet assigné au blocage ; libère la tâche si le golem est capturé ou après timeout.
`BlockingCFPBehaviour`	Manager local : demande les coûts, affecte les voisins du golem et attend les accusés de réception.

Les agents mémorisent aussi les positions alliées avec expiration temporelle. Cette mémoire sert à réduire les conflits d'exploration et à vérifier l'encerclement d'un golem. Les golems capturés sont retirés des cibles et propagés aux autres agents.

4 Choix d'exploration

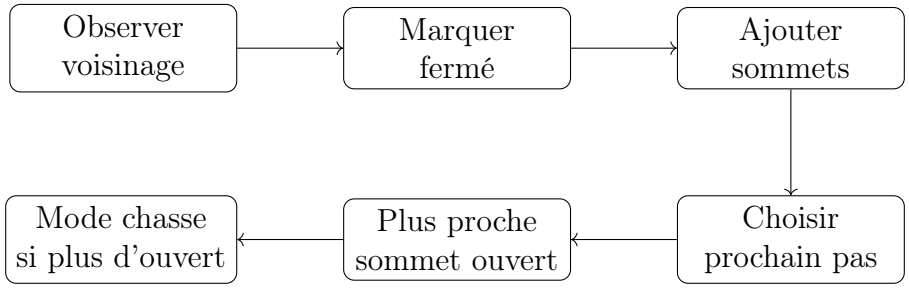

FIGURE 3 – Décision locale pendant l'exploration.

L'exploration est opportuniste. L'agent marque d'abord sa position comme fermée, ajoute les sommets voisins comme ouverts lorsqu'ils sont nouveaux, puis choisit en priorité un voisin non découvert. Si aucun voisin nouveau n'est disponible, il calcule le plus court chemin vers le sommet ouvert le plus proche, en écartant les positions connues des autres agents lorsque c'est possible.

Ce choix ressemble à une exploration de frontière avec retour par Dijkstra. Il a deux avantages : il fonctionne avec une carte partielle et il reste simple à exécuter par chaque agent. En revanche, il ne garantit pas une répartition optimale des fronts d'exploration entre agents. Deux agents peuvent encore converger vers une même zone si les informations de position sont anciennes.

TABLE 4 – Analyse des choix d'exploration

Algorithme	Forces	Limites	Complexité et arrêt
Frontière locale + voisin à découvrir	Réactif, peu coûteux, adapté à l'observation locale.	Peut produire des doublons entre agents ; dépend des positions reçues.	Temps $O(\deg(v))$ par observation. Mémoire $O(V + E)$. Communication indirecte via deltas. Arrêt local si aucun nouveau voisin.
Plus proche sommet ouvert	Ramène l'agent vers la zone non explorée la plus proche.	Optimal seulement sur la carte connue ; coûteux si relancé pour tous les ouverts.	Temps $O(F \cdot (E \log V))$ avec F sommets ouverts. Mémoire $O(V + E)$. Communication indirecte. Arrêt si plus aucun ouvert ou chemin absent.
Passage en chasse	Évite de rester bloqué en exploration.	Le seuil `getoutCnt=10` est empirique.	Critère d'arrêt : `!hasOpenNode()` ou `getoutCnt >= 10`.

Le critère de terminaison est donc pragmatique : la carte locale ne contient plus de sommet ouvert, ou l'agent a échoué plusieurs fois à sortir d'une situation de blocage. Cette stratégie est robuste en pratique, mais elle peut conclure trop tôt si la carte d'un agent est incomplète et mal synchronisée.

5 Carte, chemins et partage incrémental

`MapRepresentation` encapsule le graphe GraphStream. Les sommets ont trois états principaux : `open`, `closed` et `agent`. Les arêtes sont ajoutées au fil des observations. La classe fournit aussi Dijkstra, la liste des sommets ouverts, les sommets de degré 1, le coefficient de clustering moyen et les points d'articulation.

Le partage de carte s'appuie sur `MapDelta` : au lieu d'envoyer toute la carte à chaque cycle, l'agent transmet uniquement les nouvelles arêtes, les changements d'état de sommets et les mises à jour d'odeur. Quand la carte se stabilise, le volume de communication diminue fortement.

Table 5 – Mécanismes de carte et de chemin

Mécanisme	Forces	Limites	Complexité
Dijkstra GraphStream	Chemin exact sur la carte locale connue ; utilisé par exploration, chasse et blocage.	Ne voit pas les parties non découvertes ; recalcul fréquent.	Temps $O(E \log V)$ typique ; mémoire $O(V + E)$; communication nulle. Optimal sur carte connue. Arrêt quand destination atteinte.
Fusion de carte complète	Simple et sûre pour intégrer une carte reçue.	Plus coûteuse que le delta ; utile surtout comme resynchronisation.	Temps $O(V + E)$; mémoire $O(V + E)$; communication $O(V + E)$.
`MapDelta`	Réduit le trafic quand peu de choses changent ; inclut l'odeur.	Pas de garantie forte si un message est perdu ; l'ordre des deltas n'est pas formalisé.	Application $O(dE + dV + dS)$; communication $O(dE + dV + dS)$; arrêt si delta vide.
Hash de contenu	Permet de comparer rapidement carte et odeurs.	Un hash détecte un changement, pas sa cause.	Temps dépendant de la sérialisation ; mémoire marginale.

L'optimalité des chemins est claire mais limitée : Dijkstra donne le meilleur chemin dans la carte actuellement connue, pas dans le monde réel si des sommets manquent ou si le golem a bougé. Une amélioration serait d'ajouter une resynchronisation périodique par carte complète pour réparer les pertes de deltas.

6 Communication

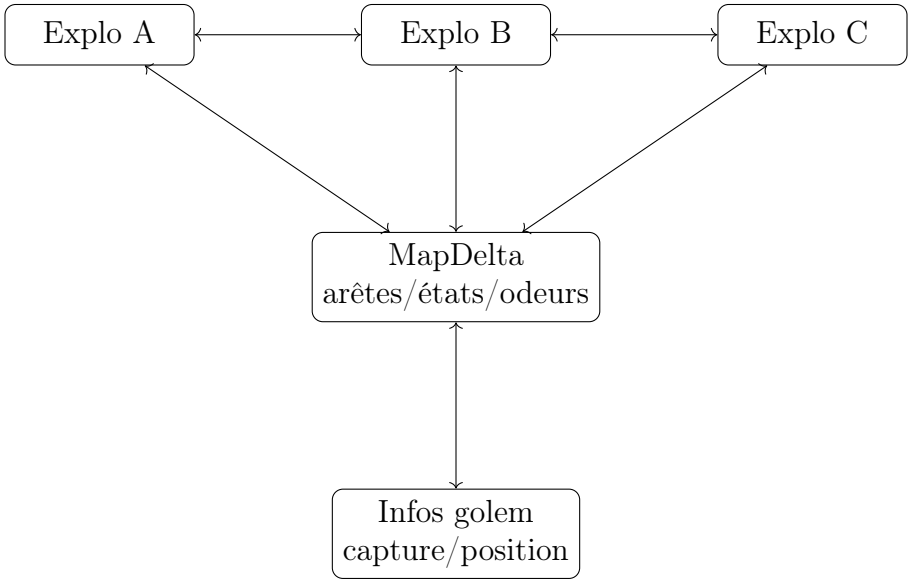

FIGURE 4 – Échanges entre explorateurs : ACL, protocoles dédiés, identifiants de message et TTL.

La communication est portée par des messages ACL avec protocoles explicites : `SHARE-POSITION`, `SHARE-STENCHD` `SHARE-INSIDESTENCH`, `GOLEM-INFO`, `CAPTURE` et `SHARE-MAP-DELTA`. Les positions et odeurs possèdent un identifiant de conversation et un TTL, ce qui limite la rediffusion et évite les boucles.

Le Directory Facilitator fournit la liste des explorateurs. La classe conserve une portée de communication dans `FSMExploAgent`, mais l'ajout effectif des récepteurs dans `SignalBehaviour` envoie actuellement aux explorateurs enregistrés plutôt qu'à un sous-ensemble filtré par distance. C'est une limite importante si l'on veut simuler strictement une portée radio.

TABLE 6 – Protocoles de communication

Protocole	Contenu	Force	Limite et coût
`SHARE-POSITION`	Sommet courant, ID, TTL.	Utile pour éviter les conflits et tester l'encerclement.	Communication $O(A)$ par cycle, ou $O(A \cdot TTL)$ avec relais ; mémoire $O(M)$ IDs. Arrêt quand TTL expire ou message déjà reçu.
`SHARE-STENCH*`	Direction d'odeur ou position interne à l'odeur.	Diffuse une piste sans connaître le golem.	Peut devenir obsolète ; arrêt quand $TTL = 1$ ou message déjà vu.
`GOLEM-INFO`	Liste des golems connus, position, timestamp, état confirmé.	Fusionne les détections et captures.	Dépend de timestamps locaux ; coût $O(G + A)$.
`CAPTURE`	Identifiant du golem capturé.	Terminaison rapide des tâches inutiles.	Best effort ; un retry simple est prévu.
`SHARE-MAP-DELTA`	Arêtes, états de sommets, odeurs.	Faible trafic après convergence.	Pas de resynchronisation forte automatique.

7 Coordination et blocage

La coordination apparaît à deux niveaux. Pendant l'exploration, les positions alliées servent à éviter de choisir un sommet ouvert déjà occupé ou visé. Pendant la chasse, la coordination devient plus explicite : lorsqu'un agent est proche d'un golem, il peut devenir manager et lancer un protocole `CFP-BLOCK` pour affecter les voisins du golem à des bloqueurs.

Le protocole est inspiré d'un mini contrat : le manager envoie un CFP contenant les voisins du golem ; les autres agents répondent avec le coût vers chaque voisin ; le manager choisit une affectation gloutonne puis envoie un award et attend des ACK. Deux timeouts bornent la décision : 1500 ms pour les propositions, 2000 ms pour les accusés.

Table 7 – Étapes du protocole de blocage

Étape	Décision	Complexité
CFP	Diffuser les voisins du golem aux explorateurs.	Communication $O(A)$; mémoire $O(N)$ voisins.
Proposition	Chaque agent calcule ses distances vers les voisins accessibles.	Temps $O(N \cdot (E \log V))$ par agent ; communication $O(N)$ par réponse.
Affectation	Choisir le meilleur coût par voisin, sans réutiliser un agent.	Glouton $O(N \cdot A \log A)$ au plus ; pas optimal comme un couplage biparti global.
ACK et blocage	Confirmer l'ordre puis passer en `MODE_BLOCKING`.	Communication $O(A)$; arrêt par tous ACK reçus ou timeout.

Cette coordination est simple et rapide, mais best effort. Elle ne garantit pas l'affectation globale minimale et reste sensible aux retards de messages. Un autre point critique est le format des messages : comme les protocoles sont encodés en chaînes, une différence de parsing peut empêcher une proposition d'être interprétée correctement. Une sérialisation structurée renforcerait cette partie.

Le blocage lui-même est local : l'agent rejoint le sommet assigné par plus court chemin, y reste tant que la cible n'est pas capturée, puis libère la tâche après capture ou timeout. Cela donne un critère d'arrêt opérationnel même lorsque tous les agents ne répondent pas.

8 Chasse au golem

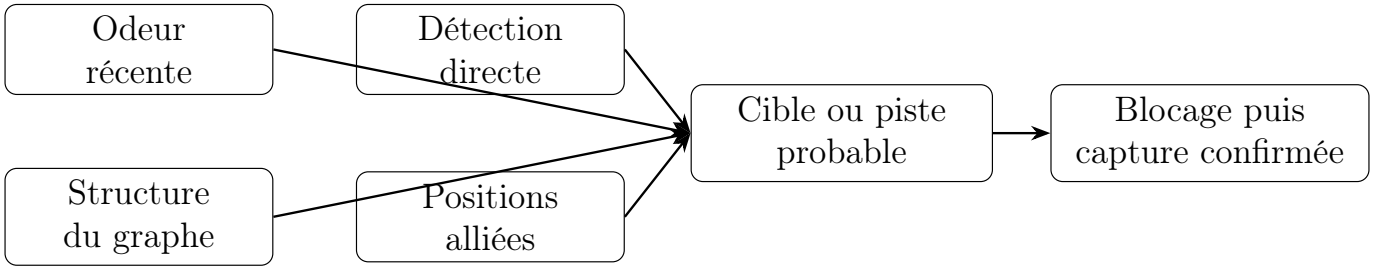

FIGURE 5 – Signaux utilisés pour la chasse.

La chasse combine plusieurs indices. Les odeurs observées sont ajoutées à une mémoire avec intensité, puis diminuent à chaque mise à jour. Une détection directe d'un agent nommé `golem` ou `wumpus` crée une information confirmée. Les informations sont ensuite partagées dans `GOLEM-INFO` avec timestamp pour propager la meilleure connaissance disponible.

Le comportement adapte aussi sa stratégie au type de graphe. Si le coefficient de clustering moyen est faible, le graphe est traité comme proche d'un arbre et l'agent privilégie les feuilles. Sinon, il exploite les points d'articulation : un agent placé sur un goulot peut réduire fortement l'espace de fuite du golem.

TABLE 8 – Algorithmes utilisés pour la chasse

Algorithme	Forces	Limites	Complexité et arrêt
Mémoire d'odeur	Filtre le bruit et conserve une piste partagée.	Seuils empiriques ; piste possiblement ancienne si le golem se déplace vite.	Temps $O(S)$ pour decay ; mémoire $O(S)$; arrêt quand intensité $\leq$ seuil.
Choix de cible	Prend le golem connu le plus proche.	Glouton, sans prédiction probabiliste.	Temps $O(G \cdot (E \log V))$; arrêt quand cible capturée ou inconnue.
Points d'articulation	Exact pour les goulots de la carte connue ; très utile sur graphes peu redondants.	Incomplet si la carte l'est ; moins utile sur graphe dense.	Temps $O(V + E)$; mémoire $O(V)$; arrêt après DFS complète.
Capture par encerclement	Critère clair : tous les voisins du golem sont occupés pendant plusieurs cycles.	Dépend de positions fraîches et de la qualité d'observation.	Temps $O(\deg(g) + A)$; communication `CAPTURE` $O(A)$; arrêt après 3 confirmations.

9 Optimalité, coûts et critères d'arrêt

Les algorithmes du projet mélangent des briques exactes et des heuristiques. Dijkstra et Tarjan sont optimaux pour leur sous-problème sur la carte connue : plus court chemin et points d'articulation. En revanche, l'allocation multi-agent, l'exploration distribuée et la poursuite du golem ne sont pas globalement optimales, car l'information est partielle et dynamique.

Table 9 – Optimalité, coûts et critères d'arrêt

Brique	Optimalité	Temps	Mémoire	Communication	Arrêt
Dijkstra	Optimal sur graphe connu non pondéré.	$O(E \log V)$	$O(V+E)$	Aucune	Destination atteinte ou chemin absent.
Exploration	Non optimale globalement ; heuristique de frontière.	$O(F \cdot (E \log V))$ au pire	$O(V+E)$	Deltas + positions	Plus d'ouvert ou `getoutCnt >= 10`.
Delta de carte	Cohérence éventuelle si messages reçus.	$O(dE+dV+dS)$	$O(V+E+S)$	$O(dE+dV+dS)$	Delta vide.
TTL + déduplication	Diffusion bornée, pas couverture garantie.	$O(M)$ traitement IDs	$O(M)$	$O(\min(A, b^{TTL}))$	$TTL = 1$ ou ID déjà reçu.
Odeur	Heuristique temporelle.	$O(S)$	$O(S)$	$O(dS)$	Intensité sous seuil.
Articulation	Exact sur carte connue.	$O(V+E)$	$O(V)$	Aucune	DFS terminée.
CFP blocage	Glouton, pas minimum global.	$O(A \cdot N \cdot E \log V)$	$O(A+N)$	$O(A \cdot N)$	Timeout ou ACK reçus.
Encerclement	Correct si positions fraîches.	$O(\deg(g) + A)$	$O(A+G)$	$O(A)$	Capture après 3 confirmations successives.

Le compromis principal est assumé : obtenir une solution distribuée réactive plutôt qu'un optimum centralisé difficile à maintenir. Les critères d'arrêt par timeout et compteurs rendent le système moins bloquant, mais ils introduisent des seuils à calibrer expérimentalement.

10 Conclusion

Le projet met en place une architecture multi-agents cohérente : un FSM explicite organise les phases, la carte locale concentre les primitives de graphe, et les messages ACL synchronisent les informations nécessaires à la coopération. Nos choix les plus solides sont la séparation des comportements, le partage incrémental par deltas, la mémoire d'odeur, la gestion de plusieurs golems et l'utilisation des points d'articulation pour transformer la chasse en problème de blocage.

Le regard critique porte surtout sur la robustesse distribuée. Les décisions restent majoritairement locales, les seuils sont empiriques, et certaines garanties dépendent de messages best effort. La portée de communication est aussi à clarifier dans l'envoi effectif des récepteurs. Enfin, l'affectation CFP gagnerait à être sérialisée de manière plus stricte et résolue par un algorithme d'appariement plutôt que par une stratégie gloutonne.

Table 10 – Extensions possibles

Extension possible	Intérêt
Affectation optimale du blocage	Remplacer l'affectation actuelle des bloqueurs par un algorithme d'optimisation, par exemple un couplage biparti pour minimiser le coût total des déplacements..
Campagne expérimentale	Comparer objectivement les performances : temps de capture, nombre de messages, taux d'échec, effet de la topologie.